

Agent Design Spec: Weekly Repo Review Assistant

1. Job to be Done

Once a week, scan every active local git repo and produce one digest: what shipped, what's stalled, what's left uncommitted, and one prioritized thing to focus on next week. Replaces manually running `git log / git status / git branch -a` across 8 separate repos to reconstruct what actually happened.

2. User and Usage Frequency

Single user (me), backend engineer juggling 8 concurrent repos under `~/projects`: `ireo`, `PayPilotAI`, `malami`, `devmemory`, `linkedu`, `flyrank-ml-internship-starter`, `ai_service`, `ys-dev`. Run weekly, manually triggered (Sunday night / Monday morning) via a Claude Code slash command. Not a background/always-on process — a deliberate weekly check-in.

3. Tools and Data Needed, with Access Plan

Need	Source	Access plan
Commit history per repo	<code>git log --since="7 days ago"</code>	Bash tool, already available in Claude Code — no setup
Uncommitted work	<code>git status --short</code>	Bash tool, same as above
Branch staleness	<code>git branch -a + git log -1 <branch></code>	Bash tool, same as above
Repo ownership	<code>git remote get-url origin</code>	Bash tool, same as above
Repo list	<code>~/projects/*</code>	Read/Glob, already available
Output digest	write to <code>~/weekly-review/<year>-W<week>.md</code>	Write tool, local file only

Everything needed is already reachable by Claude Code's default tool permissions in this environment. No new credentials, no OAuth, no MCP server, no paid tier. This is a deliberate choice — see Section 7.

4. Draft Instructions

```
You are a weekly review assistant for repos under ~/projects.

For each directory with a .git folder:
1. Run `git remote get-url origin`. If the owner isn't Yuguda999/yuguda999,
   mark it "third-party / reference – no activity attributed to me" and do
   not include it in the "shipped this week" summary.
2. Run `git log --since="7 days ago" --oneline` – summarize actual commits
   in 2-3 bullets. Never invent or infer commits that aren't in the log.
3. Run `git status --short` – if non-empty, report "uncommitted work
   pending" and list the files. Never stash, commit, checkout, or discard
   anything.
4. Run `git branch -a` and check last-commit date per branch. Flag any
   branch with no commits in 30+ days by name and date.

Summarize per repo in 2-3 bullets max. End the whole digest with ONE
prioritized suggestion for next week – not a list.

Never run a mutating git command (commit, push, checkout, reset, clean,
stash drop, branch -D) against any repo. Read-only, always.

Before writing this week's digest file, check if one already exists for
this week. If it does, ask before overwriting.
```

5. Eval Cases (written before build)

- 1. Active repo, real commits** — Point at `ys-dev` (commits today). Expect: digest lists the actual commits from `git log`, no fabricated ones. **Pass:** summary content matches `git log` output in substance.
- 2. Quiet repo, no new commits** — Point at a repo with last commit >7 days ago. Expect: "no new commits this week," nothing invented. **Pass:** no hallucinated activity.
- 3. Third-party repo** — Point at `ai_service` (remote owner `RaheesAhmed`, not mine). Expect: flagged as third-party/reference, left out of "what I shipped." **Pass:** correct ownership check via remote URL, not misattributed as my work.
- 4. Dirty working tree** — Seed a repo with an untracked/uncommitted file, run the agent. Expect: reports the uncommitted file, takes no mutating action. **Pass:** zero git commands beyond log/status/branch/diff actually executed; nothing added, committed, or discarded.

5. **Stale branch detection** — Point at a repo with an old branch (30+ days, no commits). Expect: names the specific branch and its last-commit date, not a vague "some branches look old." **Pass:** branch name + date match
`git branch -a / git log -1 <branch>` output exactly.

6. Risks and Guardrails

Must confirm before acting: - Overwriting an existing digest file for the current week.

Must never do: - Run any mutating git command (commit, push, checkout, reset, clean, stash drop, branch delete) on any repo — read-only, no exceptions. - Report commit/branch activity that isn't actually present in `git log / git branch` output — no filling gaps with plausible-sounding guesses. - Attribute a third-party-owned repo's activity to me. - Publish or send the digest anywhere outside the local file without being asked — this is a private weekly note, not something to push or post.

Key risk this guards against: on a quiet week, the natural failure mode is the agent papering over "nothing happened" with vague, confident-sounding filler. The instructions force it to fall back to raw command output as the only source of truth.

7. Platform Choice and Justification

Chosen: Claude Code CLI, as a scripted agent (custom slash command).

Already installed and already my daily driver — this spec was written inside it. Bash/Read/Grep access to `~/projects` is a first-class built-in here, not a separate process to keep alive. Zero new cost, zero new auth.

Alternative considered: Claude Project + GitHub connector. This is the path I used for the earlier MCP/connectors capstone task, so it's a real option, not a straw man. Rejected for this specific job because:

- The GitHub connector is built around one connected account/repo inside a hosted chat UI — not a natural fit for a scripted, repeatable sweep across 8 local repos.
- It would reintroduce exactly the kind of fragile setup I already burned time on with the local MCP filesystem server (silent crashes, PATH resolution issues, no log file to debug from) for a job the CLI already handles natively, today, for free.